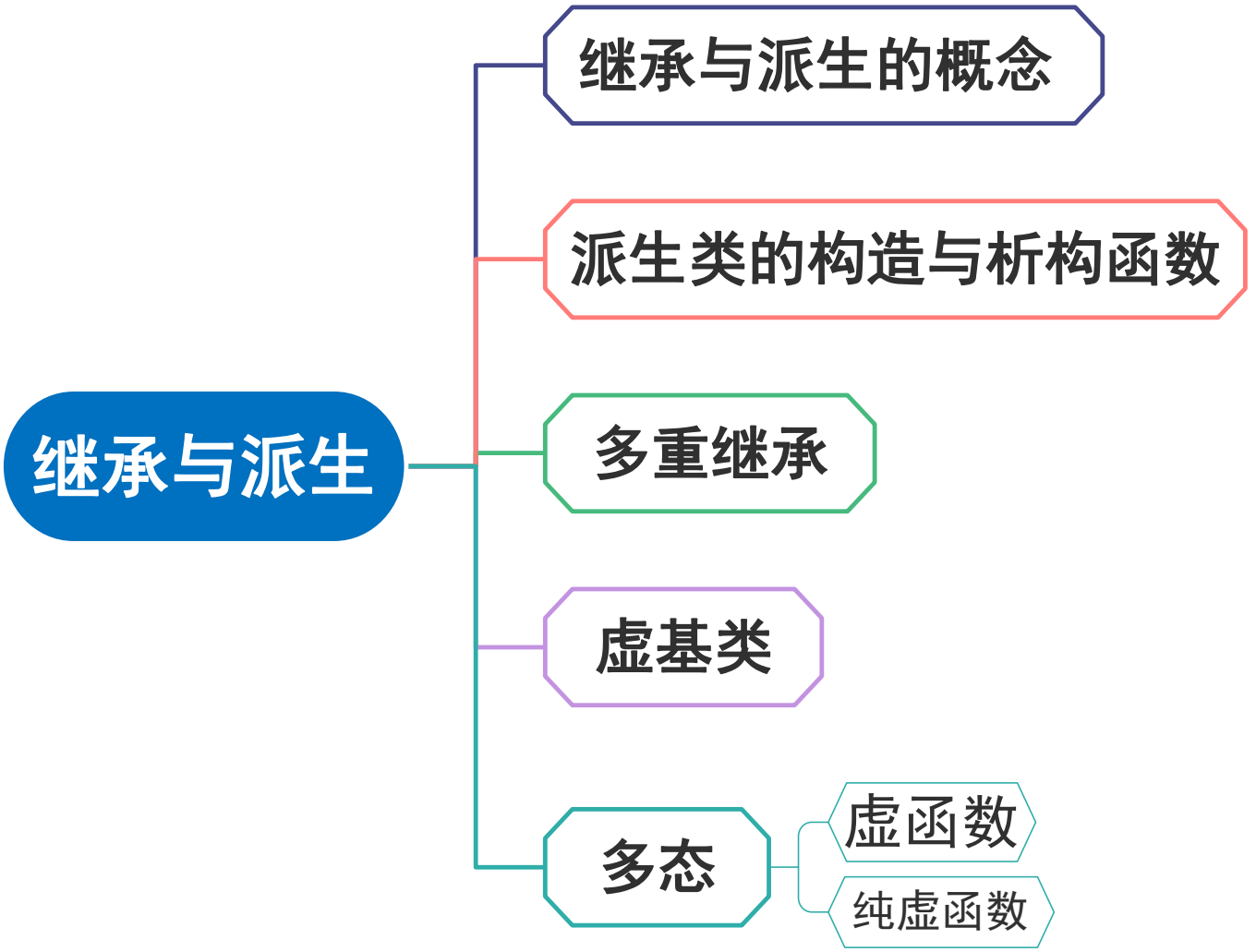
继承与多态



信息科学与工程学院
王文博 2026

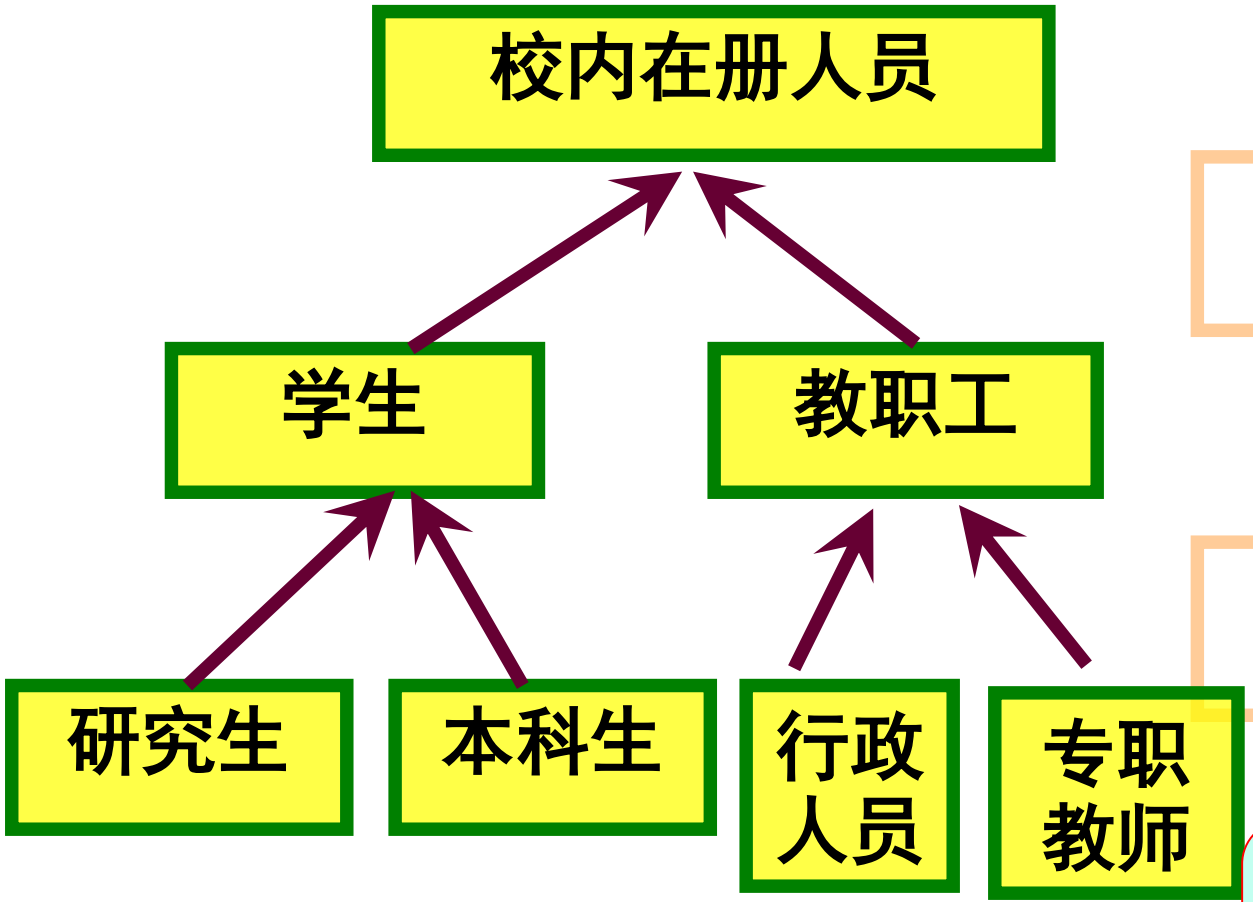
1. 继承与派生的概念

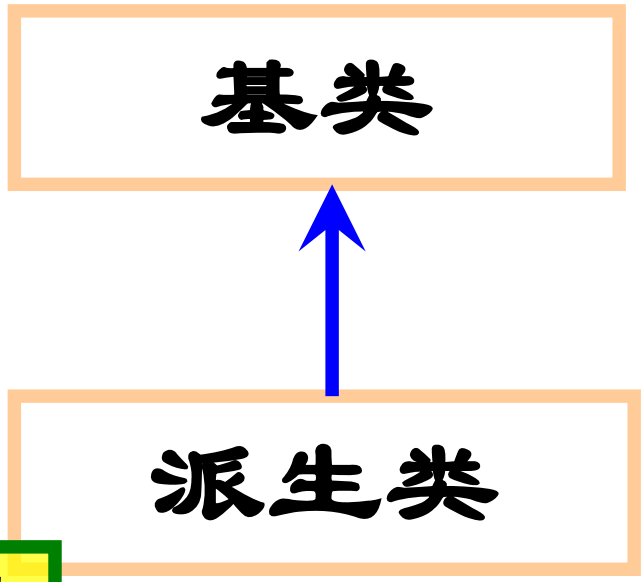
继承与派生的产生

- 在定义一个类A时，若它使用了一个已定义类R的部分或全部成员，称类A继承了类R；并称类R为基类或父类，称类A为派生类或子类。

- 一个派生类又可以作为另一个类的基类，一个基类可以派生出若干个派生类，这样便构成了类树。

1. 继承与派生的概念





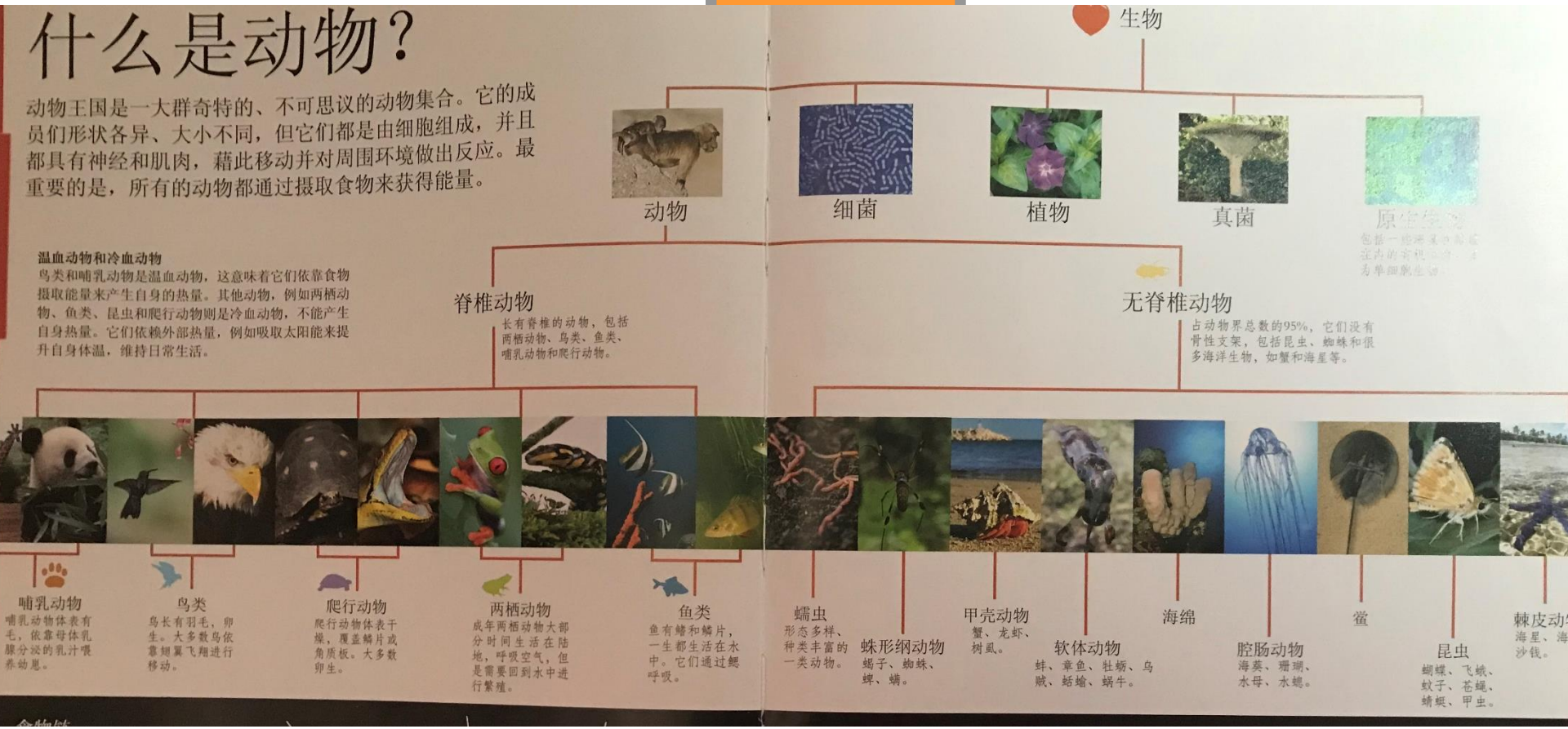
→表示继承的方向

继承机制体现了现实世界的层次结构

信息科学与工程学院
王文博 2026

1. 继承与派生的概念

动物



信息科学与工程学院
王文博 2026

1. 继承与派生的概念

- 继承的目的：实现代码重用。

- 派生的目的：当新的问题出现，原有程序无法有效解决时，需要对原有程序进行改造。

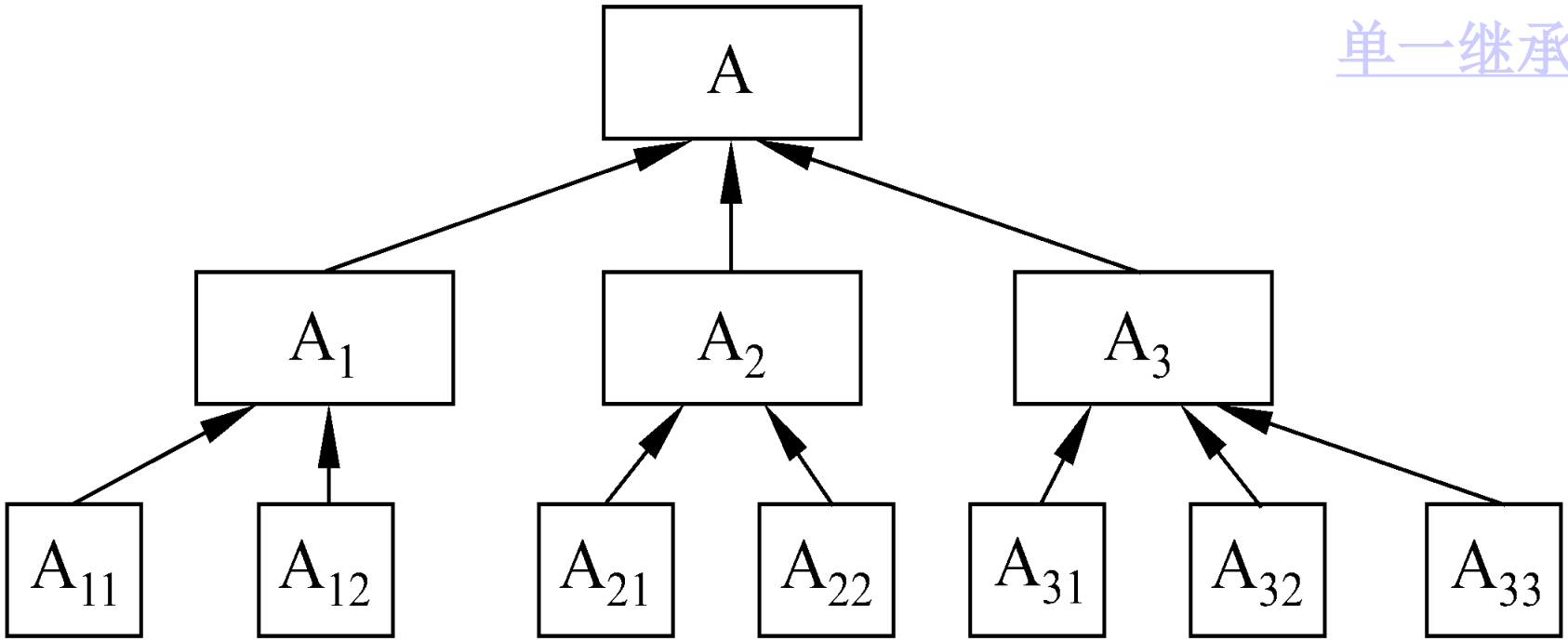
1. 继承与派生的概念

C++的二种继承方式

- 单一继承 
- 多重继承 

1. 继承与派生的概念-单继承

一个派生类只从一个基类派生，这称为**单继承**。

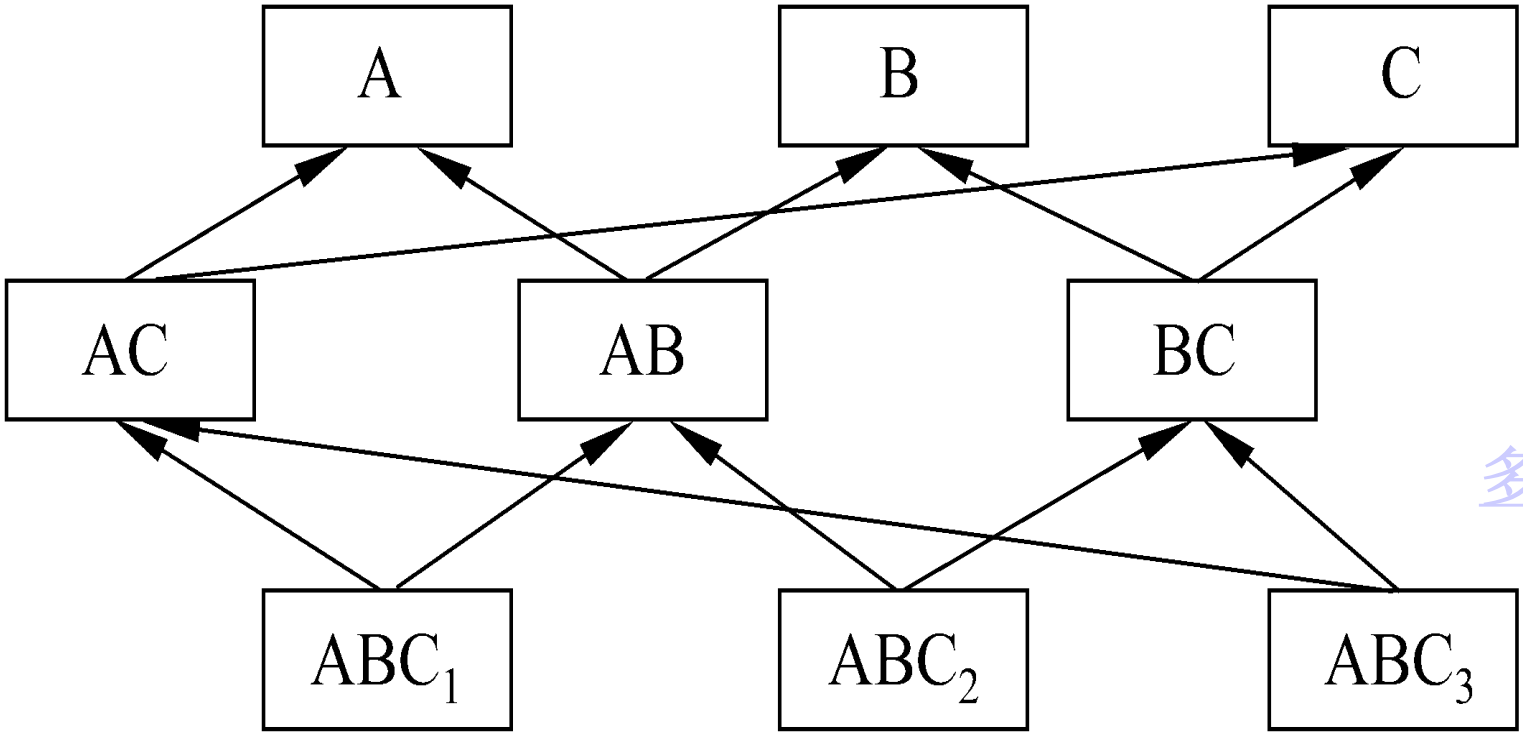


单一继承

注：箭头表示继承的方向，从派生类指向基类。

1. 继承与派生的概念-多重继承

一个派生类有两个或多个基类的称为多重继承。



多重继承

派生类是基类的具体化，而基类则是派生类的抽象。

1. 继承与派生的概念-派生的声明方式

声明派生类的一般形式为

```
class 派生类名:［继承方式］ 基类名
{
        派生类新增加的成员
};
```

1. 继承与派生的概念-派生方式

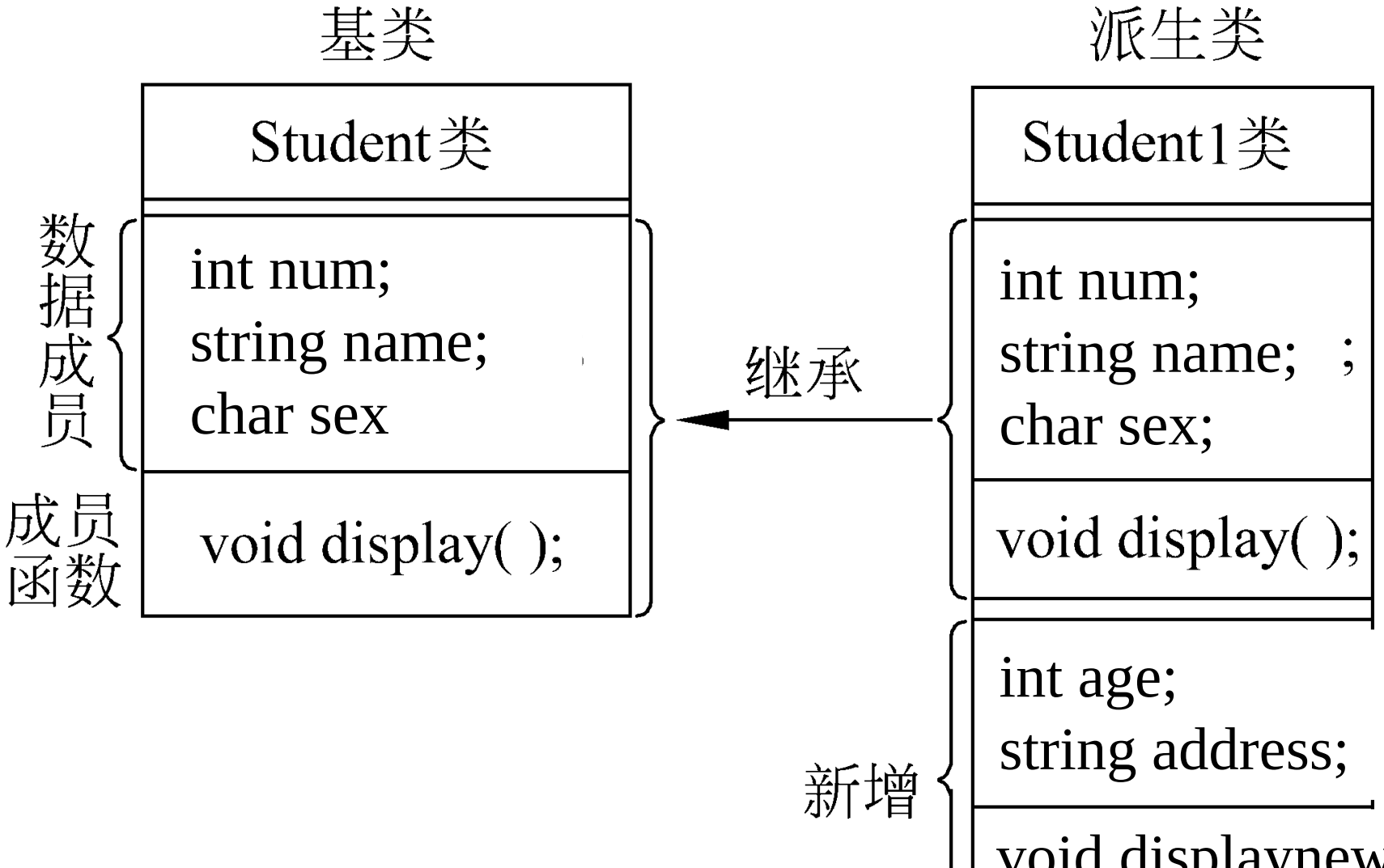
- 三种派生方式
  - –公有派生(public)
  - –私有派生(private)
  - –保护派生(protected)

若派生方式缺省，则默认为：private

- 不同派生方式的影响：
  1、派生类成员对基类成员的访问控制。【类内访问】
  2、派生类对象对基类成员的访问控制。【类外访问】

1. 继承与派生的概念-公有派生

例：若已经声明了一个基类Student，在此基础上通过单继承公有派生建立一个派生类Student1。



1. 继承与派生的概念-公有派生

【例】访问公有基类的成员

声明基类

```
class Student
 {
   public:            //基类公有成员
       void Set_value( )
       {   num=1; name="王琳" sex='w'; }
       void display( )
       {
             cout<<" num: "<<num<<endl;
             cout<<" name: "<<name<<endl;
             cout<<" sex: "<<sex<<endl;
        }
  private :           //基类私有成员
       int num;
       string name;
       char sex;
};
```

1. 继承与派生的概念-公有派生

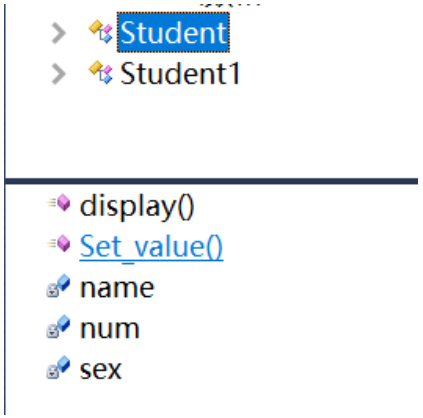
派生类名

继承方式：公有

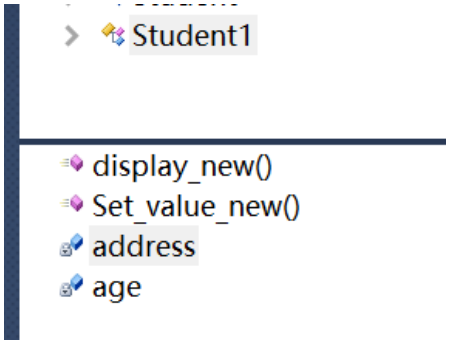
基类名

```
class Student1: public Student
{
  public:
      void display_new( )
      {

}
      void Set_value_new();
  private:
      int age;
      string addr;
 };
```



注意数据成员的属性！

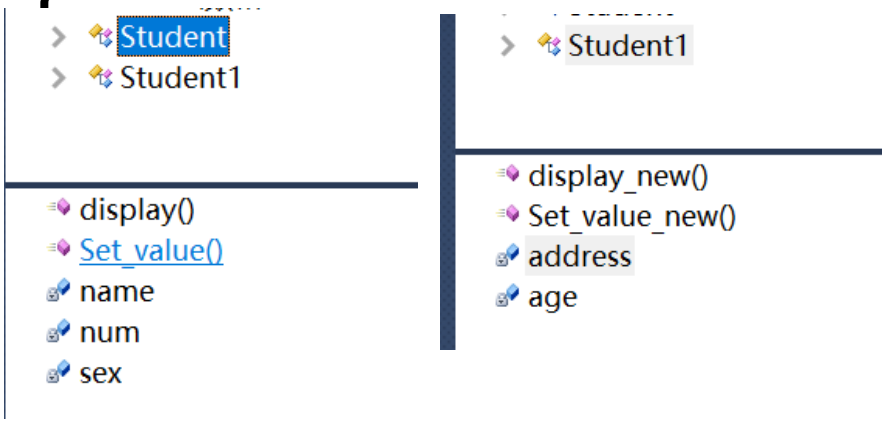


1. 继承与派生的概念-公有派生

```
int main( )
{
    Student1  stud;
    stud.Set_value( );
    stud.display( );
    stud.Setvalue_new( );
    stud.display_new( );
    return 0;
}
```

调用基类的公有成员函数，输出继承于基类的3个数据成员的值

调用派生类的公有成员函数，输出派生类中两个新增数据成员的值



如何实现主函数中只要写一行：

stud.display_1( );

即可输出5个数据？

1. 继承与派生的概念-公有派生

基类的公用成员和保护成员在派生类中保持原有访问属性，基类的私有成员在派生类中不可直接访问。

[类内访问]

派生类中的成员函数可以直接访问基类中的public和protected成员，但不能访问基类private成员。

[类外访问]

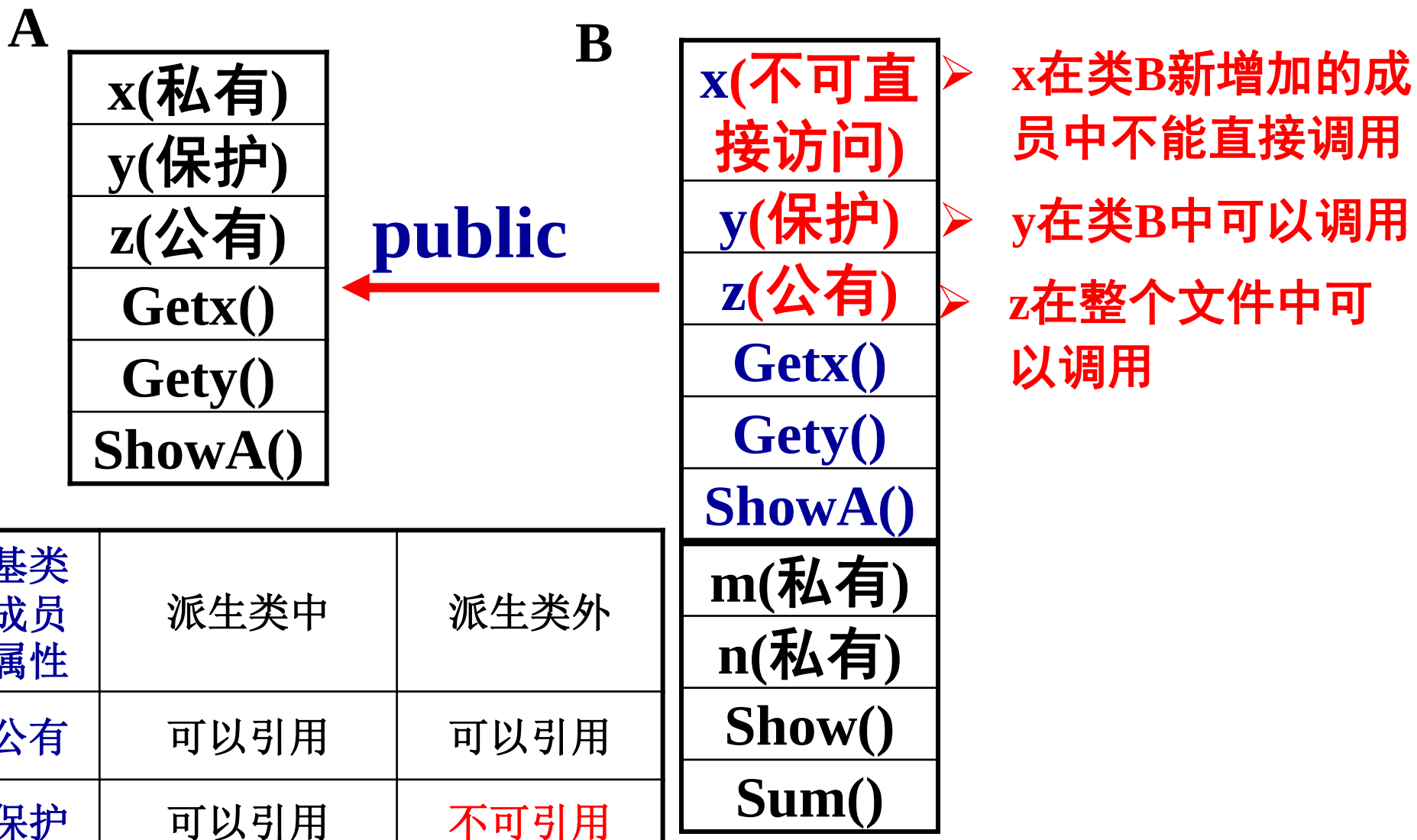
通过派生类的对象只能访问基类public成员。

1. 继承与派生的概念-公有派生

例：公有派生时，基类的保护成员派生类内可访问

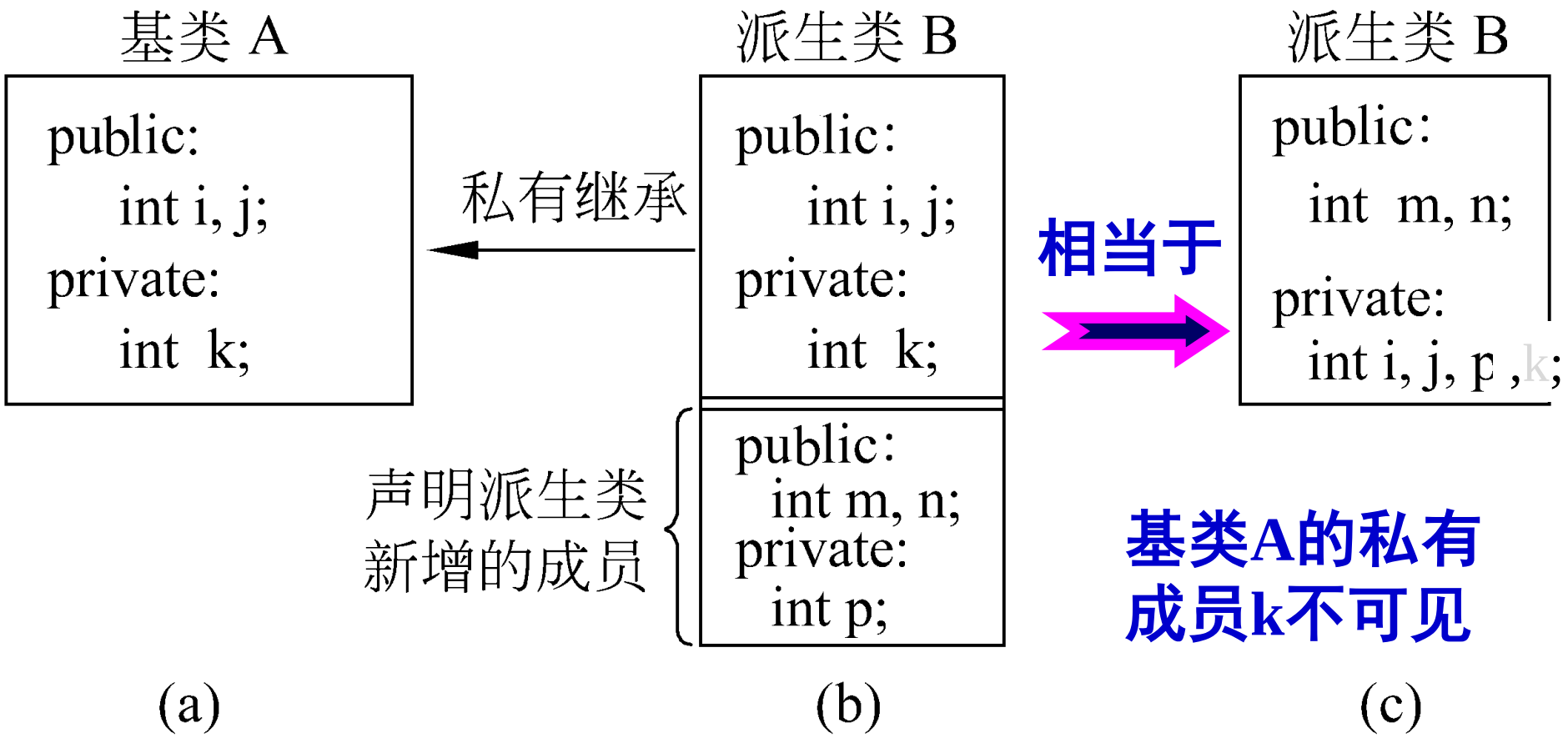
```
class A
{
    protected:
        int x;
}
class B: public A
{
public:
        void Function( );
};
void B::Function ( )
{   x=5; }
```

1. 继承与派生的概念-公有派生



基类成员属性	派生类中	派生类外
公有	可以引用	可以引用
保护	可以引用	不可引用
私有	不可直接访问	不可引用

1. 继承与派生的概念-私有派生



1. 继承与派生的概念-私有派生

例：

基类声明

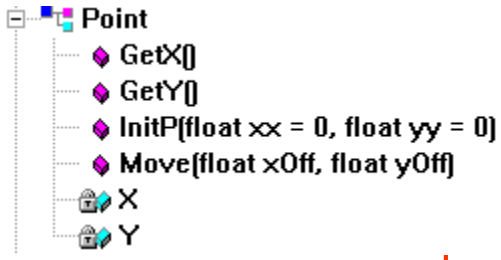
```
class Point
{
  public:
        void InitP(float xx=0, float yy=0)
        {   X=xx;      Y=yy;    }
        void Move(float xOff, float yOff)
        {   X+=xOff;      Y+=yOff;    }
        float GetX( ) {     return X;  }
        float GetY( ) {     return Y;  }
 private:
   float X,Y;
    };
```

1. 继承与派生的概念-私有派生

派生类名

继承方式：私有

基类名




```
class Rectangle: private Point
{
    public:
        void InitR( float x, float y, float w, float h)
        {      W=w;    H=h;
               InitP(x,y);    }
      void MoveR(float xOff, float yOff)
        {    Move (xOff,yOff);  }
        float GetX( ) {    return  Point::GetX( );  }
        float GetY( ) {    return  Point::GetY( );  }
        float GetH( ) {    return H;   }
        float GetW( ) {    return W;   }
private:
        float W,H;
};
```

调用基类的公有函数

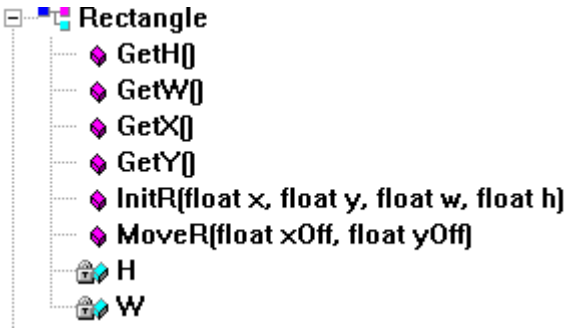
私有派生类声明

```
Rectangle rect;
rect.InitR(2,3,20,10);
```

```
X=2, Y=3, W=10, H=20
Press any key to continue
```

1. 继承与派生的概念-私有派生

基类成员属性	派生类	派生类外
公有	可以引用	不可引用
保护	可以引用	不可引用
私有	不可引用	不可引用




```
#include <iostream>
#include <cmath>
int main( )
{
    Rectangle rect;
    rect.InitR(2,3,20,10);
    rect.MoveR(3,2);
    rect.Move(3,2);
    cout<<rect.GetX()<<',' <<rect.GetY()<<','
        <<rect.GetH()<<','<<rect.GetW()<<endl;
    return 0;
}
```

Error!

1. 继承与派生的概念-私有派生

基类的公用成员和保护成员在派生类中为私有成员。基类的私有成员在派生类中不可直接访问。

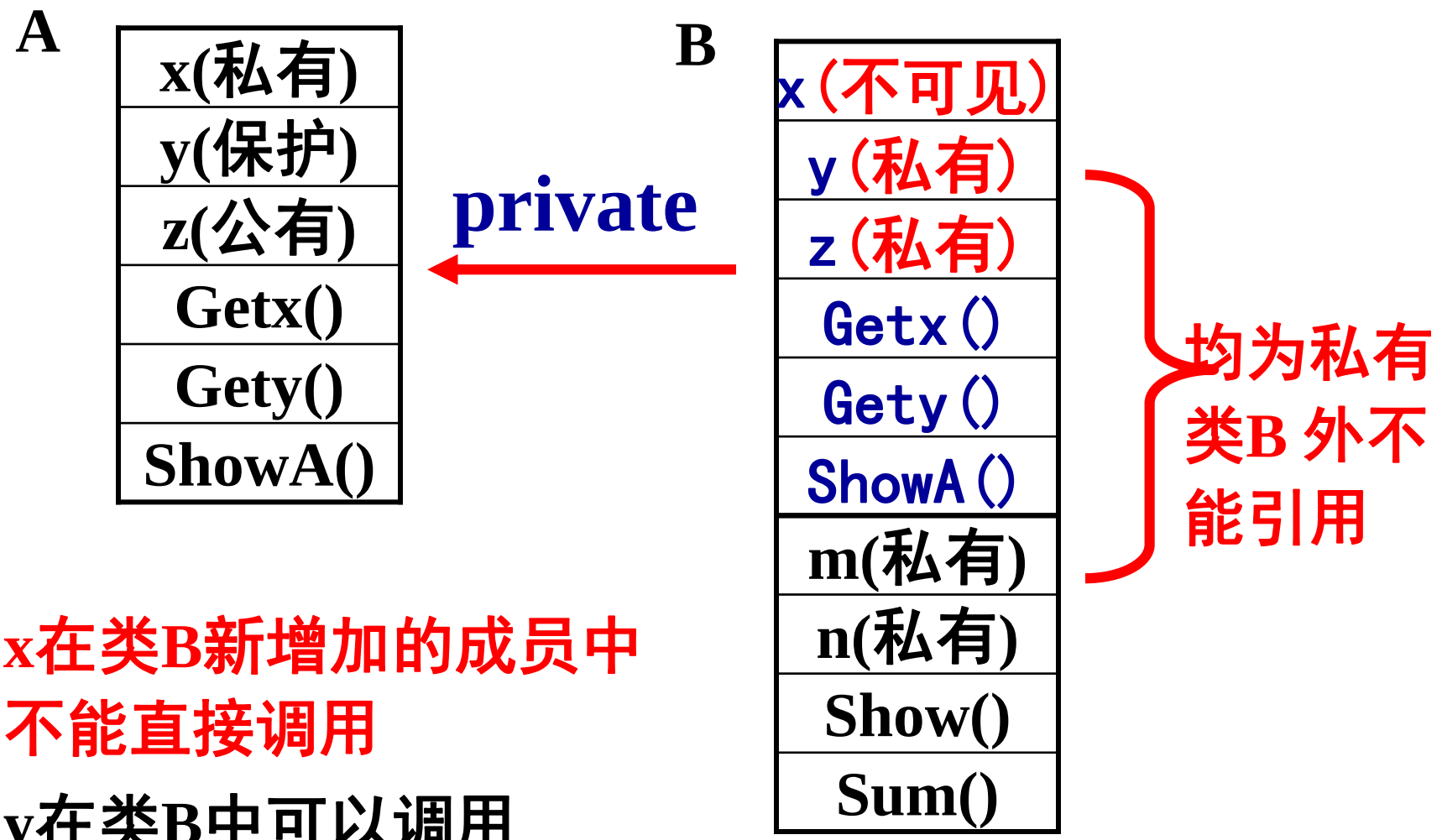
【类内访问】

基类的public和protected成员都以private身份出现在派生类中，可以被派生类中的成员函数直接访问，但不可访问基类的private成员。

【类外访问】

通过派生类的对象不能访问基类中的任何成员。

1. 继承与派生的概念-私有派生



- x在类B新增加的成员中不能直接调用
- y在类B中可以调用
- z在类B中可以调用

1. 继承与派生的概念-保护派生

基类的公有和保护成员全部成为派生类的保护成员。基类的私有成员在派生类中不可直接访问。

多层派生时体现其与私有派生的区别

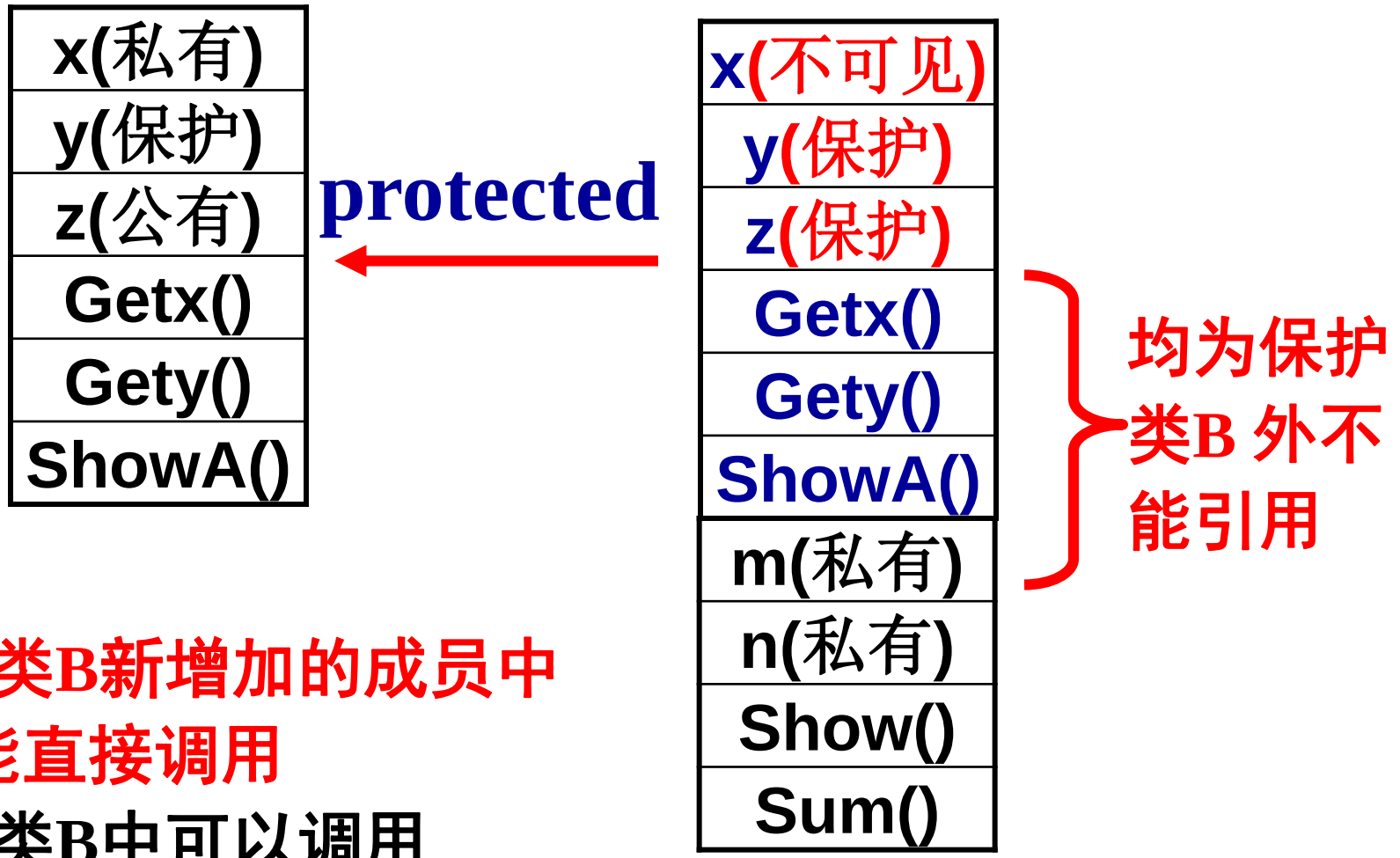
【类内访问】

基类的public和protected成员都以protected身份出现在派生类中，可以被派生类中的成员函数直接访问，但不可访问基类的private成员。

【类外访问】

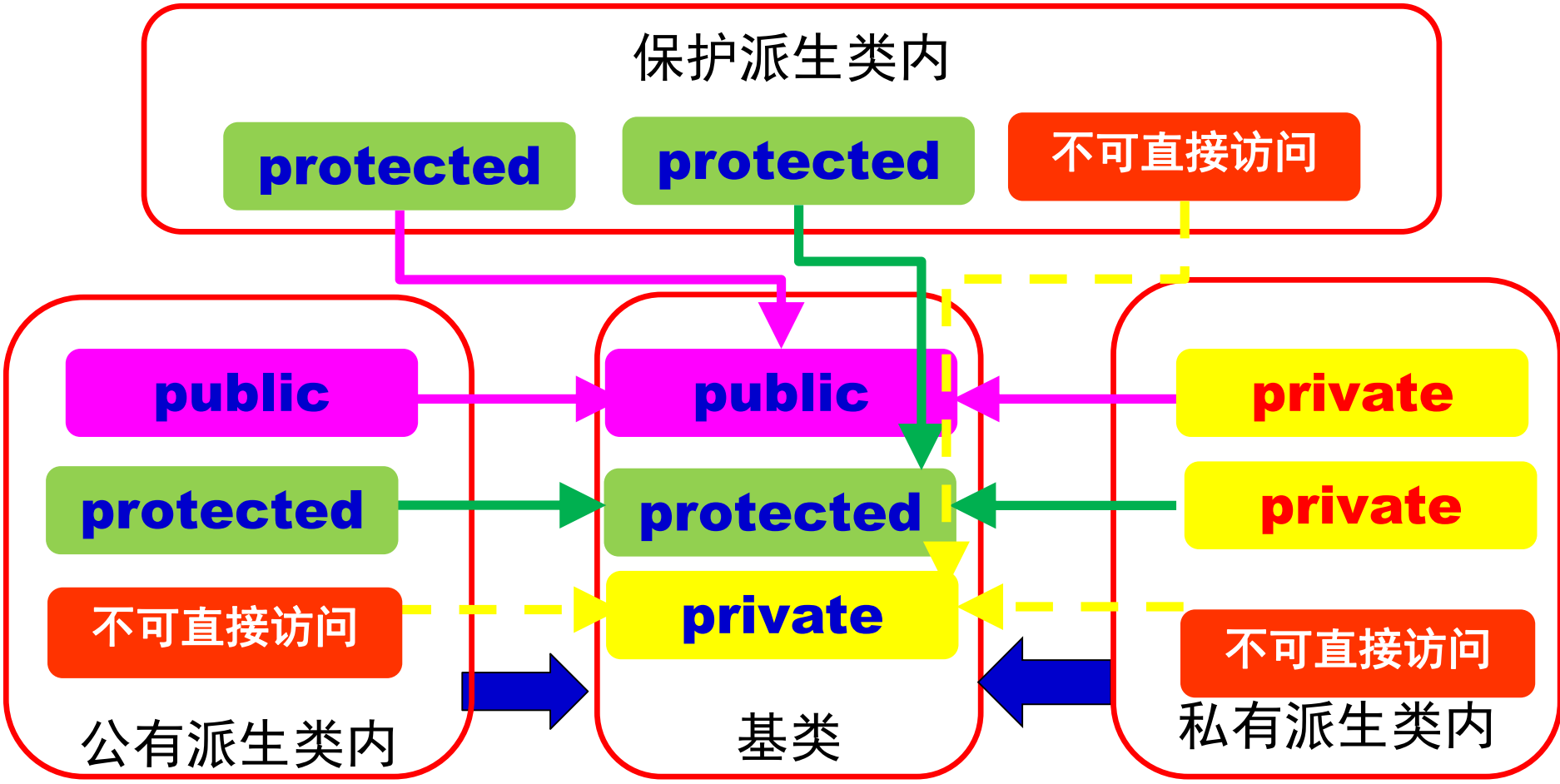
通过派生类的对象不能访问基类中的任何成员。

1. 继承与派生的概念-保护派生



- x在类B新增加的成员中不能直接调用
- y在类B中可以调用
- z在类B中可以调用

1. 继承与派生的概念-派生汇总



在派生类对象外访问派生类对象的基类成员时只有公有派生时，可对基类的public成员直接访问

```
class A
{
      int x;
   protected:
      int y;
    public:
      int z;
      void setA()
      {
         x=1;
         y=1;
         z=1;
      }
};
```

?哪些语句是错的?

```
class B: public A
{
      int a;
   public:
      int b;
      void setB()
      {
            x=1;
            y=1;
            z=1;
            a=2;
            b=2;
            c=2;
      }
protected:
      int c;
    };
```

```
int main()
{
    B b1;
    b1.setB();
    b1.x=3;
    b1.z=3;
    b1.y=3;
    b1.a=3;
    b1.b=3;
    b1.c=3;
    return 0;
}
```

```
class A
{
      int x;
   protected:
      int y;
    public:
      int z;
      void setA()
     {
          x=1;
          y=1;
          z=1;
     }
};
```

?哪些语句是错的?

```
class B: private A
{
      int a;
   public:
      int b;
     void setB()
    {
          x=1;
          y=1;
          z=1;
          a=2;
          b=2;
          c=2;
      }
protected:
      int c;
     };
```

```
int main()
{
    B b1;
    b1.setB();
    b1.x=3;
    b1.z=3;
    b1.y=3;
    b1.a=3;
    b1.b=3;
    b1.c=3;
    return 0;
}
```

```
class A
{
      int x;
   protected:
      int y;
    public:
      int z;
      void setA()
     {
        x=1;
        y=1;
        z=1;
     }
};
```

```
class B: protected A
{
     int a;
   public:
     int b;
     void setB()
    {
          x=1;
          y=1;
          z=1;
          a=2;
          b=2;
          c=2;
     }
protected:
     int c;
    };
```

```
int main()
{
    B b1;
    b1.setB();
    b1.x=3;
    b1.z=3;
    b1.y=3;
    b1.a=3;
    b1.b=3;
    b1.c=3;
    return 0;
}
```

?哪些语句是错的?

王文博 2026

小结：派生

类的成员：

数据成员和成员函数两部分。

派生类的成员：

- 从基类继承来的成员
- 在声明派生类时增加的部分

每一部分都分别包括数据成员和成员函数。

小结：派生

在派生类中，成员有4种不同的访问属性：

派生类中成员	在派生类中	在派生类外	在下层公用派生类中
公有成员	可以	可以	可以
保护成员	可以	不可以	可以
私有成员	可以	不可以	不可以
不可直接访问的私有成员	不可以	不可以	不可以

小结：派生

构造一个派生类包括3部分工作：

(1) 从基类接收全部成员。

(2) 调整从基类接收的成员。

(3) 在声明派生类时增加的成员。

要求：

根据派生类的需要慎重选择基类，使冗余量最小。

注意：

构造函数和析构函数不能从基类继承。

2、派生类的构造函数与析构函数

- 基类的构造函数不被继承，派生类中自行声明构造函数。

- 声明构造函数时，只需要对本类中新增成员进行初始化，对继承来的基类成员的初始化由基类完成。

2.1、派生类 的构造函数

定义形式：

派生类构造函数名（总参数表列）：

基类构造函数名（参数表列）

{

派生类中新增数据成员初始化语句

}

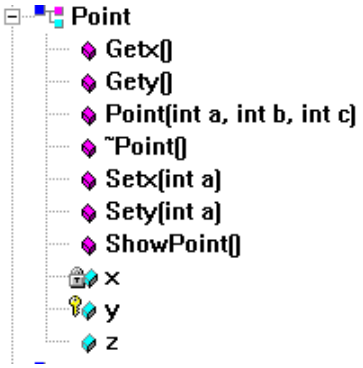
2.1、派生类 的构造函数

例：

声明基类

```
#include  <iostream>
using namespace std;
 class  Point
{
     int  x;
  protected:
     int y;
  public:
     int z;
     Point(int a,int b,int c) {     x=a;  y=b; z=c;  }
     void Setx(int a) {    x=a;    }
     void Sety(int a) {    y=a;    }
     int  Getx( ) {   return  x;     }
     int  Gety( ) {   return y;      }
     void ShowPoint( )
     {   cout<< "x="<<x<<'\t'<<"y="<<y
         <<'\t'<<"z="<<z<<'\n';   }
 };
```

2.1、派生类 的构造函数




```
class Rectangle : public Point
{
        int Length,Width;
 public:
      Rectangle(int a,int b,int c,int d,int e):
       Point(a,b,c)
      {     Length=d;
            Width=e;
       }
      void ShowRectangle( )
      {     cout<<Length<<'\t'<<Width<<'\n';
           // 输出起点坐标(x,y,z)
          }
      int Area()
      {    return  Length*Width;  }
};
```

派生类的构造函数

基类的构造函数

新增成员的初始化

声明派生类

2.1、派生类 的构造函数

```
void  main( )
{
    Rectangle r1(1,2,3,4,5);
    r1.ShowPoint( );
    r1.ShowRectangle( );
    cout<< “Area="
    cout << "y=" <<
    cout << "z="<<r1.z<<'\n';
}
```

输出：

```
x=1  y=2  z=3
4    5
1    2    3
Area=20
y=2    z=3
```

2.1、派生类 的构造函数

创建一个派生类对象时，执行构造函数的顺序

(1) 派生类构造函数先调用基类构造函数；

(2) 再执行派生类构造函数本身(派生类构造函数的函数体)。

派生类对象释放时，执行析构函数的顺序

先执行派生类析构函数，再执行其基类析构函数。

2.1派生类 的构造函数

基类构造函数调用几点注意：

- 如果基类有缺省构造函数时，可不显式调用基类构造函数，派生类的构造函数会自动调用基类缺省构造函数；
- 如果基类中有且仅有有参构造函数时，则派生类中必须显式调用基类构造函数
- 派生类构造函数显式调用基类构造函数时，派生类构造函数的参数列表里为基类构造函数提供参数。

2.1、派生类 的构造函数

声明基类

```
class B
{
    public:
        B( )
        {
            b=0;
            cout<<“Default constructor of B.\n";
        }
        B(int i)
        {   b=i;
            cout<<“Constructor of B. \n" <<endl;
        }
        ~B( ){cout<<“Destructor of B. \n"<<endl; }
        void Print( ) {cout<<b<<endl; }
        private:
            int b;
};
```


信息科学与工程学院
王文博 2026

2.1、派生类 的构造函数

声明派生类

```
class C:public B
{
   public:
     C( ) {
        c=0;
            cout<<“Default constructor of C.\n";
         }
        C(int i,int j);
        ~C( ) { cout<<“Destructor of C.\n";  }
        void Print( ) { B::Print();  cout<<c<<endl;}
  private:
          int c;
};
```

派生类 的构造函数

```
C::C(int i,int j):B(i)
{
        c=j;
        cout<<"Constructor of C."<<endl;
}
```

派生类构造函数

```
int main( )
{
        C obj(5,6);
        obj.Print( );
        return 0;
}
```

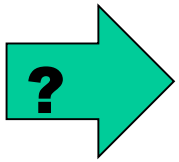
?

输出：

```
Constructor of B.
Constructor of C.
5
6
Destructor of C.
Destructor of B.
```

派生类 的构造函数

```
int main( )
{
        C obj;
        obj.Print( );
        return 0;
}
```



2.2 有子对象的派生类 的构造函数

类的数据成员中包含类的对象:对象中的对象--子对象

定义派生类构造函数的一般形式:

派生类构造函数名(总参数表列):

基类构造函数名(参数表列),子对象名(参数表列)

{

派生类中新增数成员据成员初始化语句

}

2.2 有子对象的派生类 的构造函数

例 包含子对象的派生类的构造函数。

```
#include <iostream>
#include <string>
using namespace std;
class Student
{
public:
        Student(int n, string nam )
        {   /*基类构造函数*/ }

        void display( )
        { /*成员函数，输出基类数据成员*/ }


protected:
       int num;
    string name;
};
```

声明基类

2.2 有子对象的派生类 的构造函数

```
class Student1: public Student
{
    Student monitor;
    int age;
    string addr;
public:
    Student1(int n, string nam,int n1,  string nam1,int a, string ad):
        Student(n,nam), monitor(n1,nam1)
            {    age=a; addr=ad;   }
    void show( )
    {        /*输出num, name, age和addr*/    }
    void show_monitor( ) //成员函数
    {  /*调用基类成员函数monitor.display( );输出子对象*/ }
};
```

定义子对象

派生类构造函数？

基类成员构造用基类名，

基类子对象构造用对象名

基类构造函数

子对象构造函数

声明派生类

2.2 有子对象的派生类 的构造函数

```
int main( ) {
    Student1 stud1(10010,"Wang-li", 10001,"Li-sun", 19,
    "115 Beijing Road,Shanghai");
    stud1.show( );
     stud1.show_monitor();
    return 0;
}
```

输出:

```
This student is:
num: 10010
name: Wang-li
age: 19
address:115 Beijing Road,Shanghai
Class monitor is:
num:10001
name:Li-sun
```

小结：派生类 的构造函数

派生类(含子对象)构造函数的任务

（1） 对基类数据成员初始化；

（2） 对子对象数据成员初始化；

（3） 对派生类数据成员初始化。

执行顺序

①调用基类构造函数，对基类数据成员初始化；

②调用子对象构造函数，对子对象数据成员初始化；

③执行派生类构造函数本身，对派生类数据成员初始化。

小结：派生类 的构造函数

派生类构造函数的参数内容

- 派生类构造函数的总参数表列中的参数，应包括基类构造函数和子对象的参数表列中的参数。

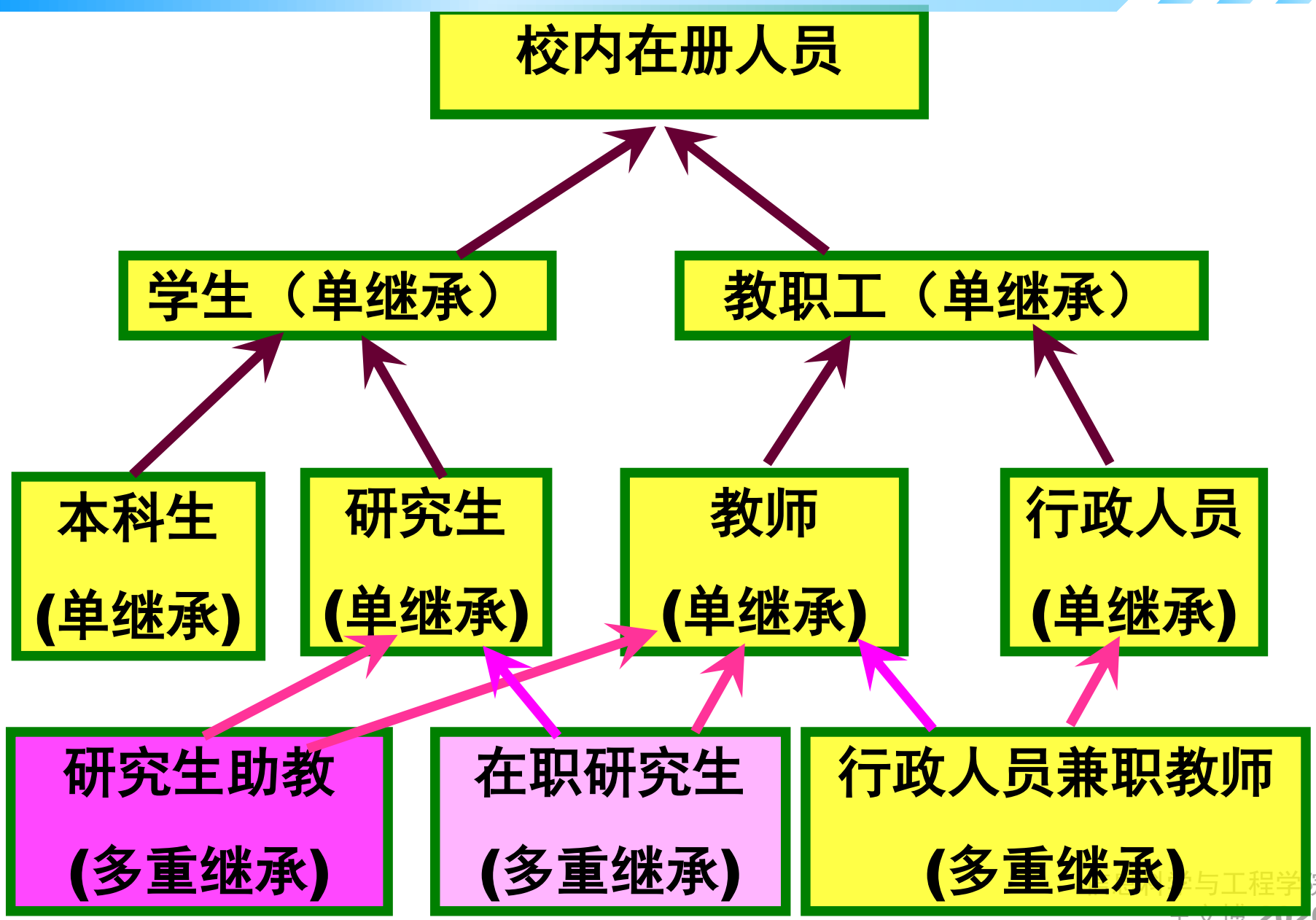
- 基类构造函数和子对象的次序可以任意,习惯上一般先写基类构造函数。

- 如果有多个子对象，派生类构造函数的写法依此类推，应列出每一个子对象名及其参数表列。

3 多重继承与派生类的成员标识

- 单继承
  - 派生类只从一个基类派生。
- 多重继承
  - 派生类从多个基类派生。
- 多重派生
  - 由一个基类派生出多个不同的派生类。
- 多级(层)派生
  - 派生类又作为基类，继续派生新的类。

3 多重继承与派生类的成员标识

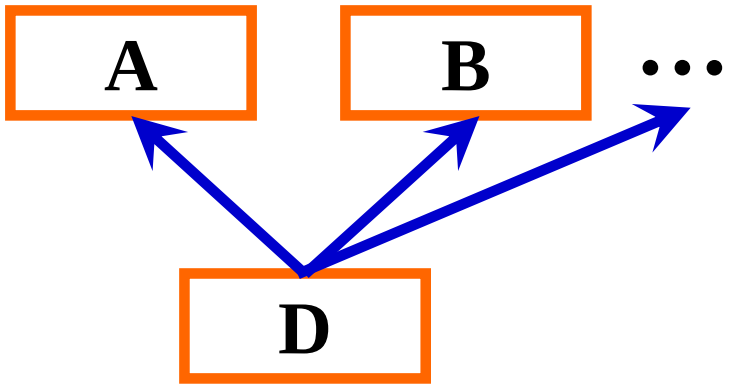


与工程学院
2026

3 多重继承与派生类的成员标识

定义：

一个派生类有两个或多个基类，派生类从两个或多个基类中继承所需的属性。



3 多重继承与派生类的成员标识

声明多重继承的方法

用多个基类派生一个类的一般格式为：

```
class  类名: [继承方式]基类名1,
              [继承方式]基类名2,
              ...,
              [继承方式]基类名n
{
        派生类新增加的成员
};
```

3 多重继承与派生类的成员标识

```
class D:public A, private B, protected C
{
        类D新增加的成员
};
```

D分别从基类A、B、C的继承的成员的访问权限？

D的构造函数？

```
class A
{
      int x;
   protected:
      int y;
    public:
      int z;
      void setA()      {
         x=1;
         y=1;
         z=1;
         }
};
```

```
class B:
{
      int a;
   public:
      int b;
     void setB(){
           a=2;
           b=2;
           c=2;
           }
protected:
      int c;
     };
```

```
class C:
{
      int i;
   public:
      int j;
     void setC(){
           i=3;
           j=3;
           k=3;
           }
protected:
      int k;
     };
```

```
class D:public A, private B, protected C
{       int d;
   public:
        void  setD(){
};
```

这里可以对哪些变量赋值？

多重继承派生类的构造函数

一般形式：

派生类构造函数名(总参数表列):

基类1构造函数(参数表列),

基类2构造函数(参数表列),

基类3构造函数(参数表列)

{ 派生类中新增数据成员成员初始化语句 }

各基类的排列顺序任意。

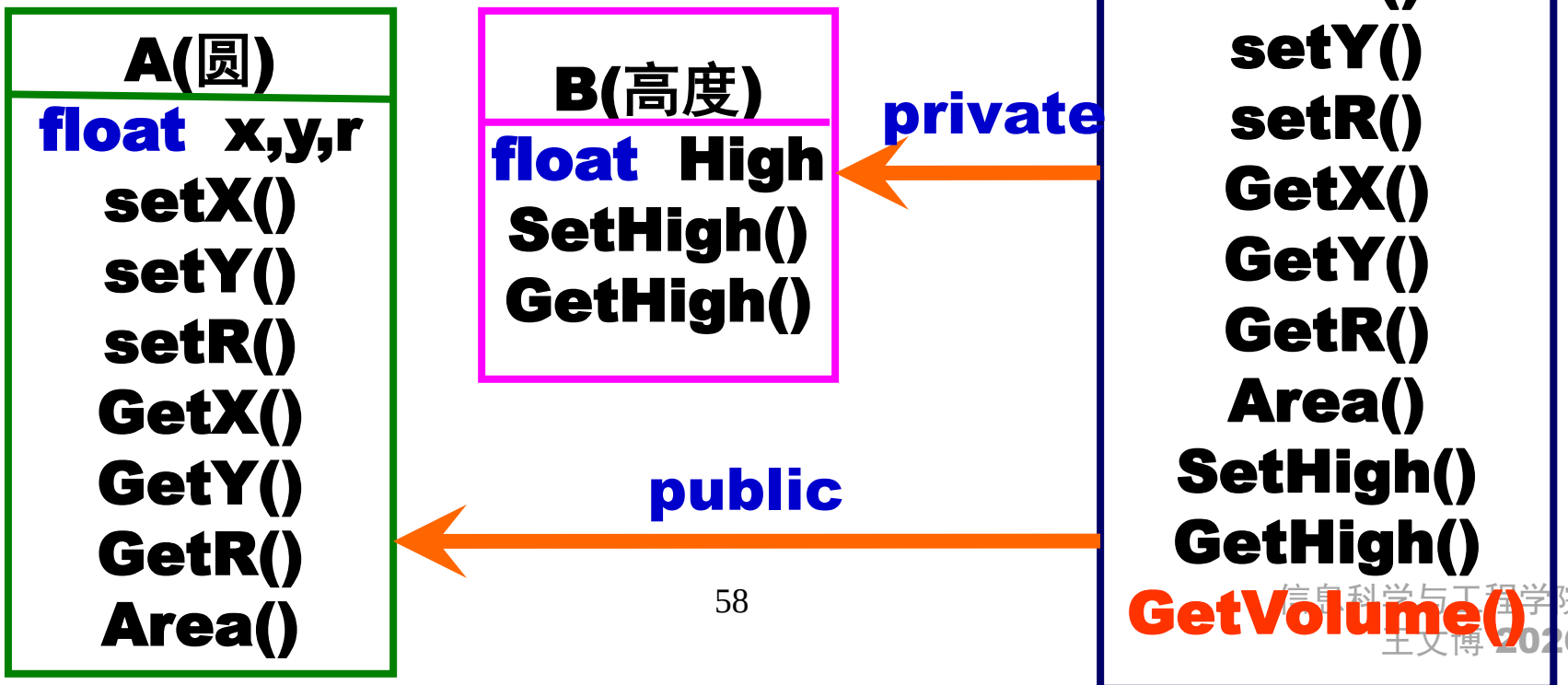
多重继承派生类的构造函数

派生类构造函数的执行顺序

- 先调用基类的构造函数，再执行派生类构造函数的函数体。

- 调用基类构造函数的顺序是按照**声明派生类时基类出现的顺序**。

例:类A描述了一个圆，
类B描述高度，
类C由二个基类A（公有派生）和B（私有派生）派生而来，描述一个圆柱体，包含这二个类的所有成员，并增加了描述圆柱体体积的新成员Volume。



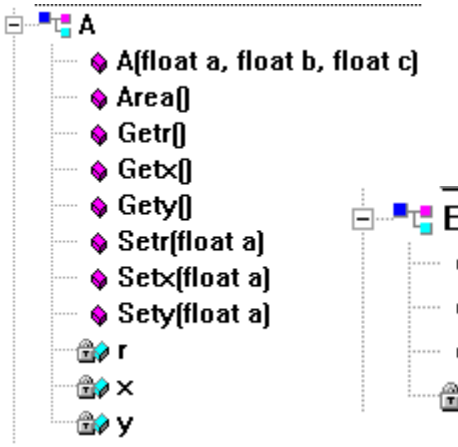
信息科学与工程学院
王文博 2026

多重继承派生类的构造函数

```
#include <iostream>
using namespace std;
class A
{
   float x,y,r;
public:
   A( float a, float b, float c)
   {  x=a;y=b;r=c; }
   void Setx(float a) {  x=a; }
   void Sety(float a) {  y=a; }
   void Setr(float a) {  r=a;  }
   float Getx( ) {  return x;  }
   float Gety( ) {  return y;  }
   float Getr( ) {  return r;  }
   float Area( ) {  return (r*r* 3.14159);
};
```

```
class B {
   float High;
public:
   B(float a)
{   High=a; }
   void SetHigh(float a)
{  High=a;  }
   float GetHigh( )
 {  return High ;  }
};
```

多重继承派生类的构造函数




```
class C:public A,private B
{
    float Volume;
  public:
                构造函数？

    float GetVolume( ) {   return Volume;   }
};
```

多重继承派生类的构造函数

```
int main()
{
    A a1(6,8,9);   //a
    B b1=23;
    C c1(1,2,3,4);
    cout << a1.Getx( )<<'\t'<<a1.Gety( )<<'\n';
    cout << a1.Getr( )<<'\t'<<a1.Area( )<<'\n';
    cout <<b1.GetHigh( )<<'\n';
    cout <<c1.GetVolume( )<<'\n';
    return 0;
}
```

若去掉a行，将所有a1变为c1，结果为何？

输出：
x=6　　y=8
r=9　　area=245.469
High=23
Volume=113.097

输出：
x=1　　y=2
r=3　　area=28.2743
High=23
Volume=113.097

计算机科学与工程学院
王文博 2026

多重继承派生类的构造函数

例:输出派生类中构造函数与析构函数的调用关系

```
#include  <iostream>
using namespace std;
class  Base1
{
            int  x;
 public:
    Base1(int a)
    {
       x=a;
      cout<<"调用基类1的构造函数!\n";
    }
    ~Base1( )
   {    cout<<"调用基类1的析构函数!\n";
   }
};
```

```
class  Base2
{
            int y;
 public:
    Base2(int a)
    {
       y=a;
      cout<<"调用基类2的构造函数!\n";
    }
    ~Base2( )
    {   cout<<“调用基类2的析构函数!\n”;
     }

};
```

多重继承派生类的构造函数

```
class Derived: public Base2, public  Base1
{
            int z;
public:
          Derived(int a,int b):Base1(a),Base2(20)
          {   z=b;                cout<<"调用派生类的构造函数!\n";      }
          ~Derived( ) {     cout<<"调用派生类的析构函数!\n";
};
void  main( )
{       Derived   c(100,200);          }
```

输出结果？

```
class Derived:public Base1, public  Base2 {
          int z;
          Base2  b1,b2;
 public:
   Derived(int a,int b):Base1(a),Base2(20), b1(200),b2(a+b)
          {   z=b;   cout<<"调用派生类的构造函数!\n";   }
   ~Derived( ){cout<<"调用派生类的析构函数!\n";}
};
void  main(void)
{          Derived   c(100,200);}
```

输出结果？

fortest

```
class A {
public:
    int a;
   A(){  a=1;cout<<"1"<<'\t';}
   A(int x){a=x;cout<<a<<'\t';}
  ~A(){cout<<'~'<<a<<'\n';}
};
```

```
class B2: public A {
    A  a0;
public:
int b;
  B2(){b=5;cout<<"5"<<'\n';}
   B2(int x,int y):A(x)
     {b=y;cout<<y<<'\n';}
   ~B2(){cout<<'~'<< b<<'\t';}
};
```

```
class C {
public:
    int c;
   C(){ c=2;cout<<“2"<<'\t';}
   C(int x){c=x;cout<<c<<'\t';}
  ~C(){cout<<'~'<<c<<'\n';}
};
```

```
class D: public B2,public B1{
      int d;
     C ct,cs;
  public:
    D(){d=7;cout<<"7"<<'\n';}
    D(int x,int y,int z,int w):B1(x),B2(x,y),
       cs(z),ct(w)
        { d=y;cout<<y<<'\n';}
     void seta(int x,int y)
     {B2::a=x;  B2::b=y;}
      ~D(){cout<<'~'<<d<<'\t';}
};
```

```
class B1: public A {
public:
     int b;
   B1(){b=3;cout<<"3"<<'\n';}
   B1(int x){b=x;cout<<x<<'\n';}
    ~B1(){cout<<'~'<<b<<'\t';}
};
```